COMPUTER NETWORKS · FINAL PROJECT

Passive TLS Fingerprinting with JA3/JA3S and JA4/JA4S

Identifying browsers, curl and custom clients from the unencrypted TLS handshake, without decrypting anything

Vadithya Harsha Vardhan Nayak
23110349

Sailaja Mudu
23110288

Banoth Arvindh
23110058

https://github.com/HARSHA219219/Passive-TLS-Fingerprinting-with-JA3-JA3S-and-JA4-JA4S.git

Problem Statement & Objectives

THE PROBLEM

Most web traffic is encrypted, so a monitor cannot read payloads. But the **ClientHello** and **ServerHello** are sent in clear text, and every TLS library fills them differently.

Goal: passively identify the client application or library behind each TLS connection from its handshake fingerprint alone, without decryption.

Input: packets, live interface or pcap

Output: JA3, JA3S, JA4, JA4S + verdict with evidence

Out of scope: QUIC/HTTP3, decryption

OBJECTIVES

- **01** Passively capture ClientHello and ServerHello, live and from pcap
- **02** Compute JA3/JA3S per spec; JA4/JA4S as stretch goal
- **03** Validate against published reference fingerprints
- **04** Curated database identifying at least 5 distinct clients
- **05** Tell curl vs browser vs custom client apart on the wire
- **06** Explain security use (C2 detection) and limits (randomisation, spoofing)

Architecture & Mechanism

Capture
Scapy + Npcap, live or pcap

→

TCP reassembly
reorder, retransmit, any port

→

Hello parser
Client/ServerHello, GREASE removed

→

Fingerprints
JA3, JA3S, JA4, JA4S

→

Identify
lookup in SQLite reference DB

How the fingerprints are built

JA3 = MD5 of version, ciphers, extensions *in wire order*, groups, point formats

`JA4 t13d1517h2_8daaf6152771_cb7bf5808d99`

TLS 1.3 · SNI · 15 ciphers · 17 extensions · h2 _ hash of sorted ciphers (the library) _ hash of sorted extensions + signature algorithms

Evidence-based verdict

JA4 and JA3 are looked up separately

Identified: agree · ja4_only · ja3_only

Not identified: ambiguous · conflict · unknown, never guessed

Extensions & Issues Fixed

What we built on top

Evidence statuses: JA3/JA4 conflicts shown, not hidden

Published validation: FoxIO test pcaps, Salesforce vectors, Wireshark

Curated DB with sources: 9 learned clients + 16 published (Tor, Trickbot, IcedID, Sliver…)

New clients: Firefox, an unknown Python 3.14, JA3 and JA4 spoofers

One-command experiments + step-by-step demo script

Optional: Linux XDP source, Redis/Valkey sync, TRex profile (not evaluated)

Bugs found and fixed

JA4 silently overrode a disagreeing JA3 → **conflict status**

IPv6 captures with macOS loopback framing not decoded → **fallback decoder**

Crash on packets without an IP length → **safe decode**

Firefox used HTTP/3 and a shared profile → **fresh profile, QUIC off**

Wikipedia shown as “Sliver C2 server” → **server match shown as a hint**

README claimed 20/20; tool printed 20/24 → **measured numbers only**

Evaluation Results

75/75
held-out connections of 9 clients identified (JA4 + JA3)

44/75
with JA3 alone: Chrome 0/12, Edge 0/19

19 → 1
Chrome: 19 connections gave 19 JA3s but 1 JA4

10/10
spoofed hellos accepted as Chrome

5/5
unknown client reported as UNKNOWN

Validation	Published / FoxIO reference	Wireshark 4.6.7	Note
JA3	2/2 Salesforce examples	225/225	exact
JA3S	1/1 (explained)	202/202	publisher hashed a newline
JA4	135/136 (1 explained)	225/225	FoxIO spec vs code differ
JA4S	76/76	not in Wireshark	exact

Non-Functional Testing Parameters

Performance

Parse 50–97 µs, JA3+JA4 62–103 µs, identify 4–31 µs per hello; 1,900–3,400 packets/s offline (one core, 2 runs)

Scalability

64 KiB cap per flow, stops after first hello; 50,000 streams max, 120 s idle expiry. Bottleneck is Scapy decoding

Resource use

Peak Python memory 0.9 MB on the 6,658-packet test capture; SQLite needs no server

Reliability

Malformed or truncated hellos dropped, never a crash; reordering, retransmission, overlap, IPv6, any port

Testing & reproducibility

34 automated tests (Redis test skips without a server); every result regenerated by one script

Security & privacy

Passive only, no decryption; unknowns never guessed; background traffic filtered from shared captures

Not measured: line-rate capture. Linux XDP and a TRex profile are provided as extensions only.

Challenges Faced

Chrome’s hello spans 2 TCP segments

Post-quantum key share makes it too big for one packet → our own TCP reassembly

Browsers shuffle extensions

New JA3 on every connection since Chrome 110 → JA4 sorts first, plus evidence statuses

GREASE in signature algorithms

Wireshark’s JA4 changes per connection → followed the spec, confirmed by FoxIO’s test pcap

Browsers prefer QUIC/HTTP3

UDP is invisible to a TCP sniffer → headless runs with QUIC off and fresh profiles

References disagree

Salesforce hash includes a newline; FoxIO spec vs code on ALPN → tested each, documented

Ground truth & privacy

Labels from per-run time windows and the owning process; personal traffic filtered out

Takeaway: a fingerprint is strong evidence of the TLS library, not proof of identity: it can be spoofed and it drifts between versions.